

TDM Insight

A native Android vancomycin therapeutic drug monitoring calculator

Mowlid Haibe
Abdinaazir Mustafe
Elham Ahmedngus

Albukhary International University
Lecturer: Ts. Mohd Zulkifli Mohd Zaki
Semester 3, 2025/2026

Academic prototype — not a clinically validated system. All demonstration cases are fictional.

What we will cover

1 • The problem

Why vancomycin dosing is calculated by hand today, and what goes wrong.

2 • The solution

What TDM Insight does, for whom, and what it deliberately leaves out.

3 • Architecture

Layered MVVM, the component map and the technology stack.

4 • Behaviour

Use cases, activity, sequence and data-flow models.

5 • The engine

Three pharmacokinetic pathways, validation, and worked examples.

6 • Evidence

Interface, testing, a known defect, limitations and conclusion.

01

The problem

Vancomycin has a narrow therapeutic index and is still dosed with a paper worksheet.

A calculation that has to be right

Vancomycin is first-line therapy for MRSA, and its effective dose sits close to its toxic dose.

Too little

Treatment failure. The MRSA infection is not cleared and resistance is encouraged.

Too much

Nephrotoxicity. Exposure above the target band is associated with acute kidney injury.

Highly individual

Clearance depends on renal function, which varies enormously between patients. A fixed mg/kg dose is not adequate.

Rybak et al. (2020) moved monitoring from trough alone to **AUC₂₄**, target **400–600 mg·h/L** — clinically better founded, but it cannot be read off a single laboratory value.

Four weaknesses in current practice

Hospital pharmacy performs these calculations on paper worksheets or a general-purpose calculator.

- **Method selection is left to memory.** Which equations apply depends on which samples the laboratory returned. Nothing on a worksheet enforces the right choice.
- **There is no input validation.** A worksheet accepts a serum creatinine of 9.8 $\mu\text{mol/L}$ as readily as 98. A transposed decimal reaches the dose recommendation silently.
- **Cross-field inconsistencies go undetected.** A trough recorded as drawn before the peak yields a negative elimination rate constant — meaningless, but still displayed.
- **There is no audit trail.** Worksheets are discarded, so no one can review how a past recommendation was reached.

Who this is for

Clinical pharmacist

Needs to turn a laboratory report into an AUC_{24} estimate and a dose recommendation at the bedside — and to justify it afterwards.

Pharmacy student

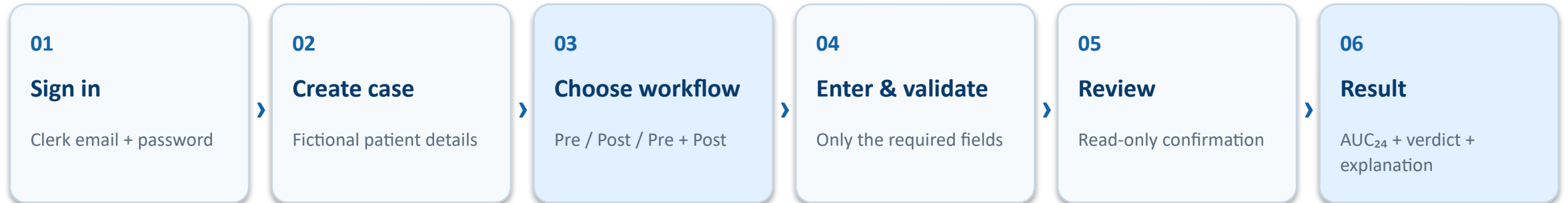
Needs to see which equation produced which intermediate value, not just the final answer.

Prospective user

Reaches the project through the public landing page and wants to understand it before installing.

No user of any kind may enter real patient data. The restriction is surfaced on first launch, on every input screen, and in the About screen.

TDM Insight in one flow



The whole calculation path is **on-device and synchronous**. Persistence and identity are optional layers on top — remove the credentials and the calculator still works.

Objectives

Each objective maps to a mandatory requirement of the lecturer's case study.

#	Objective	Delivered as
1	Native Android application in Kotlin	Jetpack Compose + Material 3, 18 screens, minSdk 26
2	All three vancomycin sampling workflows	<code>calculatePre</code> · <code>calculatePost</code> · <code>calculatePrePost</code>
3	Dynamic input forms	Field groups rendered conditionally on the selected workflow
4	Validation beyond emptiness	Range, cross-field and math-safety checks; errors vs. warnings
5	Engine separated from the UI	<code>domain/engine/VancoEngine.kt</code> — no Android imports
6	Intermediate and final results	$k_e \cdot t_{1/2} \cdot V_d \cdot V_d/kg \cdot CL \cdot AUC_{24} \cdot dose \cdot C_{min} \cdot C_{max}$
7	Explainable results	Four-phase Explanation screen with substituted formulas
8	Material 3 light and dark themes	Two hand-authored colour schemes, selectable in Settings

Main features

Three PK pathways

Cockcroft–Gault, Newton–Raphson fit, and Sawchuk–Zaske two-point regression.

Dynamic forms

Only the fields the chosen workflow needs are ever rendered.

Two-tier validation

Red blocks the calculation; amber advises and does not.

Live CrCl feedback

Cockcroft–Gault recomputed on every keystroke on the patient screen.

Explainable output

Four phases: inputs → ke derivation → parameters → final result.

Per-user case history

Twenty most recent saved cases, from PostgreSQL.

3

sampling workflows

9

PK values per result

18

application screens

31

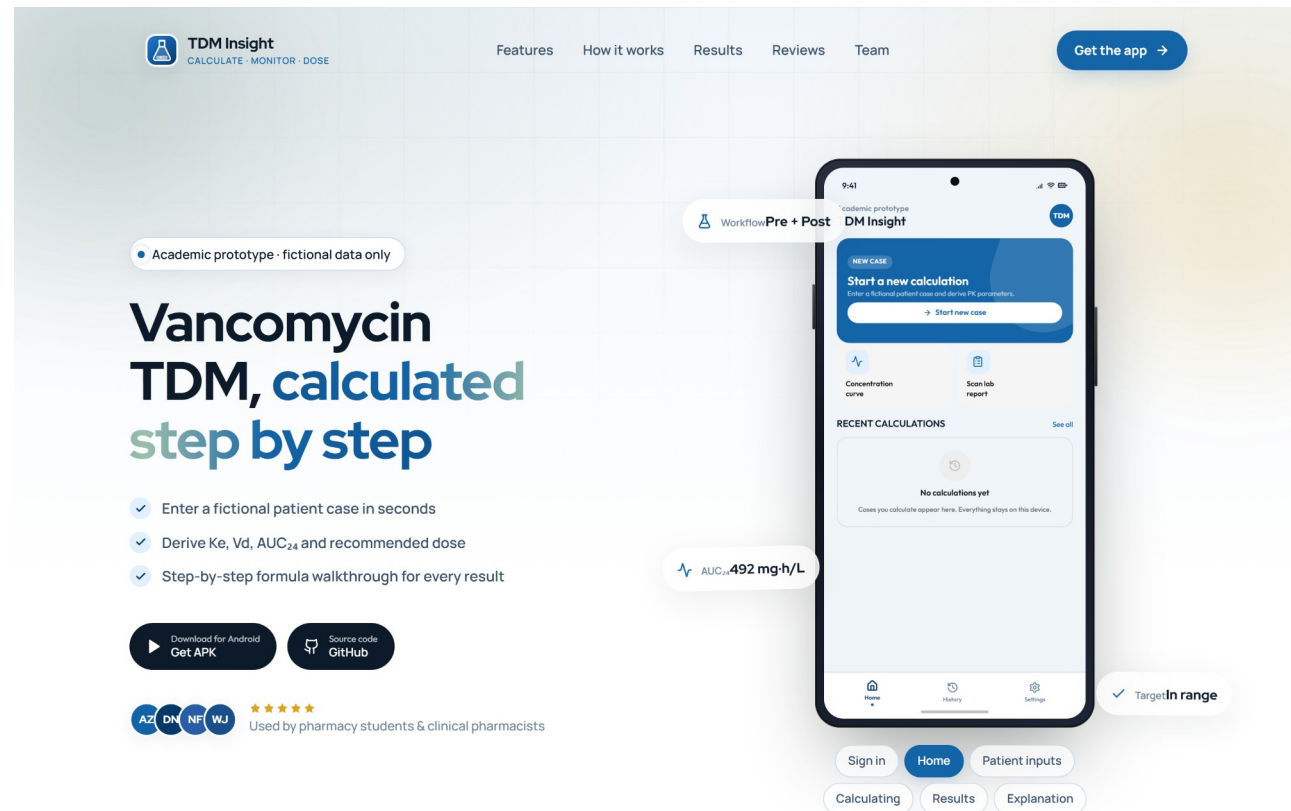
unit tests

2

database tables

The landing page

A static React site that introduces the product before installation.



Hero section — proposition, headline capabilities and download links

Technology stack

Every version below is read from the project's own build files.

Android

Kotlin 2.0 · Jetpack Compose (BOM 2024.08) ·
Material 3 · Navigation Compose 2.7.7 ·
Lifecycle ViewModel 2.8.4 · Coroutines 1.8.1 ·
compileSdk 35, minSdk 26

Networking & data

Ktor 2.3.12 with the OkHttp engine · kotlinx-
serialization 1.7.1 · supabase-kt `postgrest-kt`
2.6.1 · hosted PostgreSQL with PostgREST

Identity

Clerk Frontend API called directly over REST ·
session JWT persisted in private
SharedPreferences · publishable key injected
through `BuildConfig`

Landing page

React 19.1 · Vite 6.3 · TypeScript 5.9 · Tailwind
CSS 4.1 · React Router 7.6 · lucide-react · built
with Bun

Testing

JUnit 4.13.2 — 31 unit tests over the engine
and the validators, run with
`./gradlew :app:test`

Tooling

Android Studio · Gradle Kotlin DSL · Git and
GitHub · Supabase SQL editor · Claude Code as
a declared AI assistant

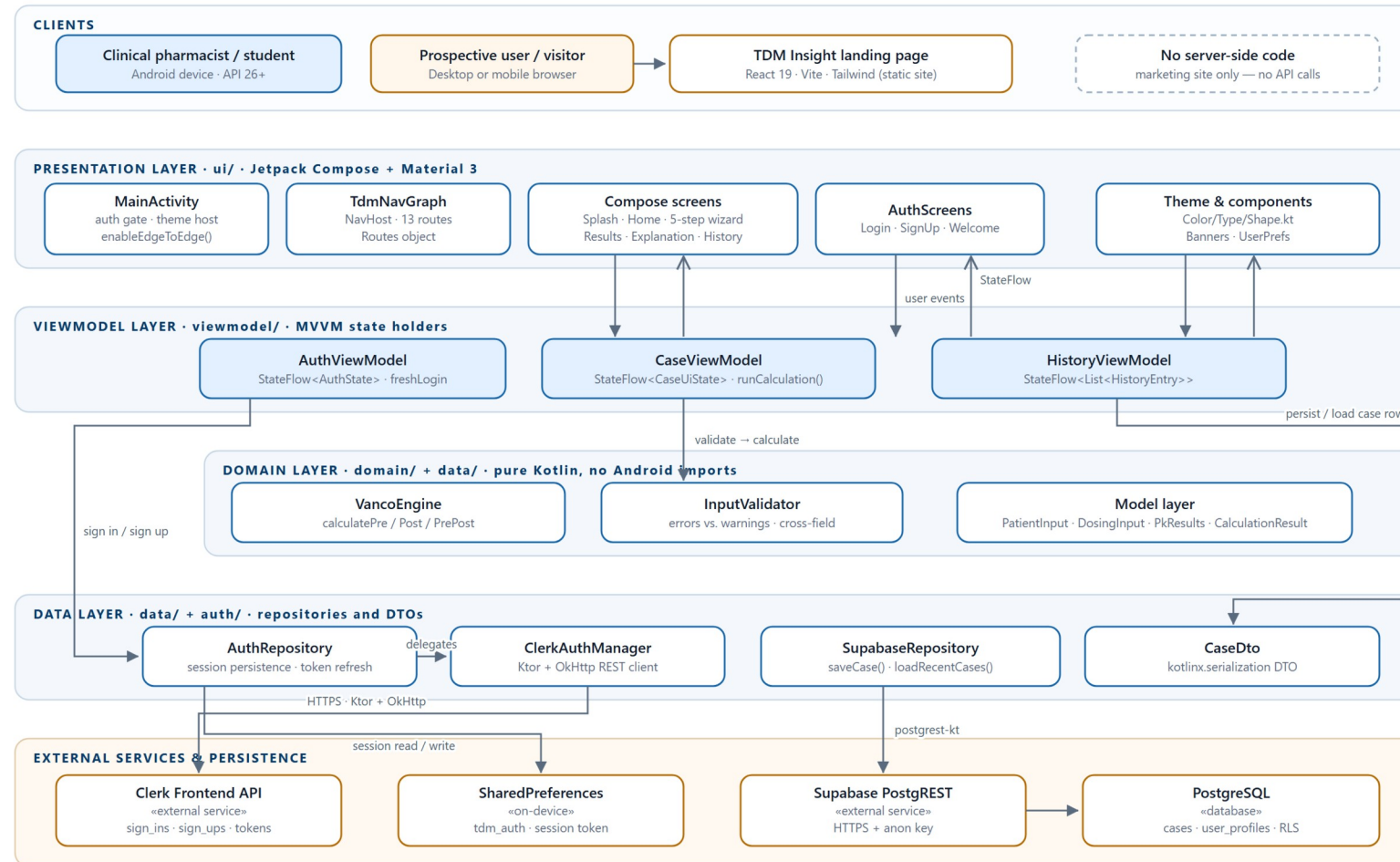
The landing page makes **no** network call to any backend. `apiClient.ts` is unused template scaffolding and is not reachable from `App.tsx` — the architecture diagram records this.

02

System architecture

Layered MVVM, with the calculation engine isolated from everything Android.

Layered system architecture



Solid arrows are synchronous calls; open arrows are reactive state emissions

Why MVVM

Testability of the engine

A layer with no Android dependency can be exercised by plain JVM unit tests in under a second — no emulator.

Natural fit with Compose

`StateFlow` consumed through `collectAsState()`; recomposition is driven by state, not by imperative view updates.

Survives rotation

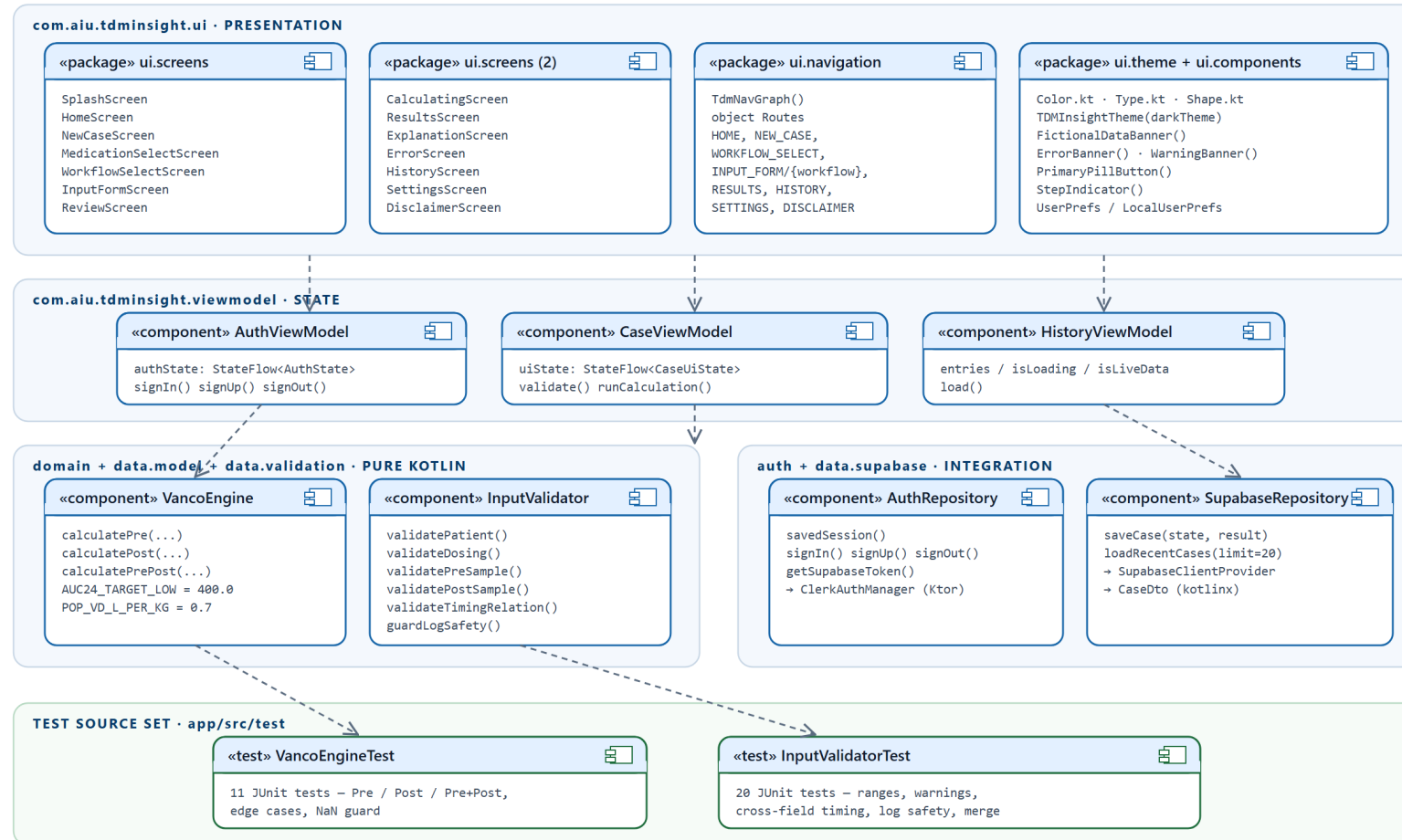
ViewModels outlive activity recreation, so the five-step wizard never loses the user's input.

No massive activity

A single activity of roughly 260 lines whose only jobs are theming and the authentication gate.

Dependency injection is manual: `TdmApplication` holds four lazy singletons. For four objects that is simpler and more defensible in a viva than adding Hilt or Koin.

Component and package structure



Dashed open arrows are «use» dependencies. The domain layer has no dependency on any Android or integration component, which is what makes it unit-testable in isolation.

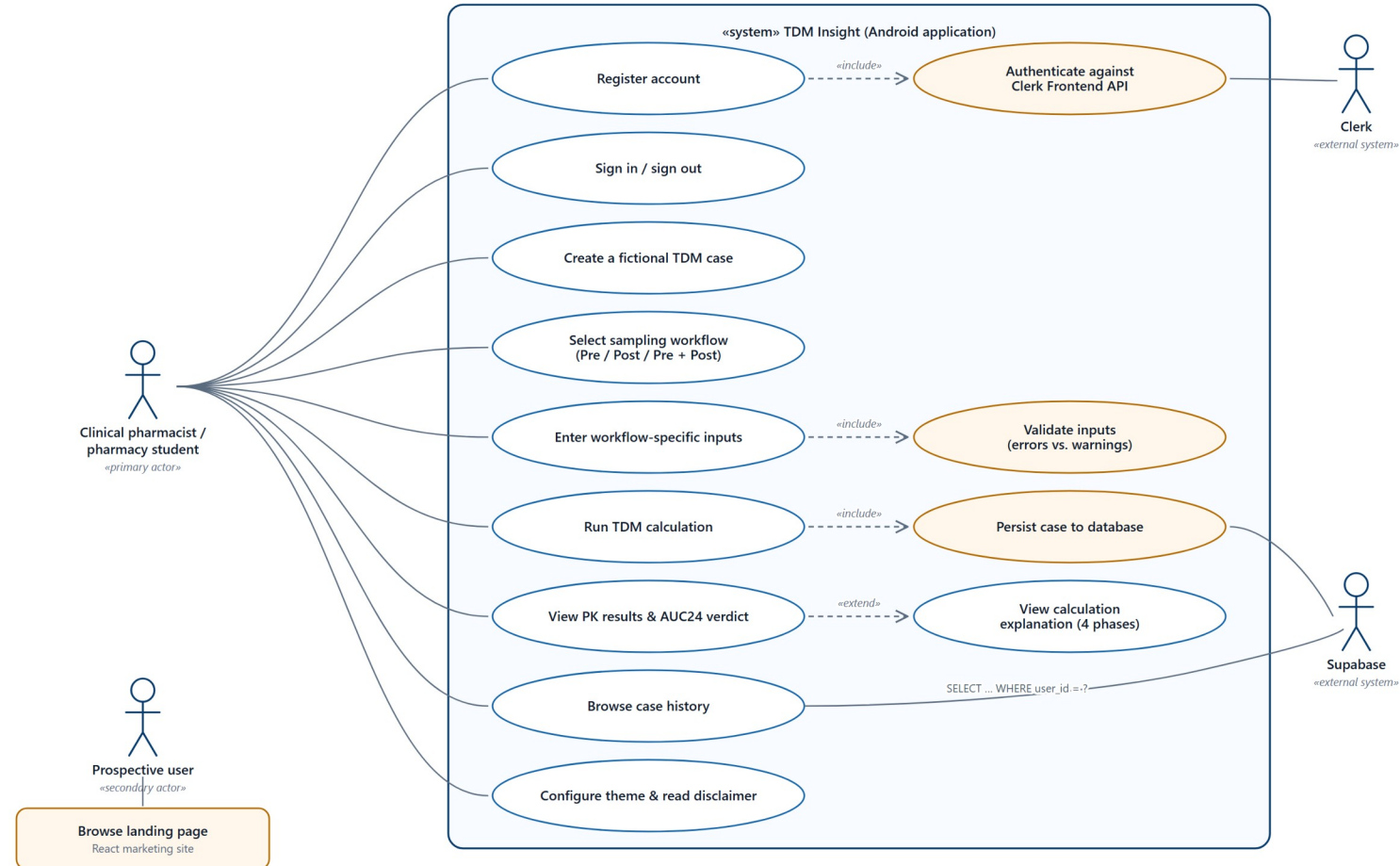
No arrow enters the domain layer from the data or presentation layers

03

How the system behaves

Use cases, activity, sequence and data-flow models — all traced from the source.

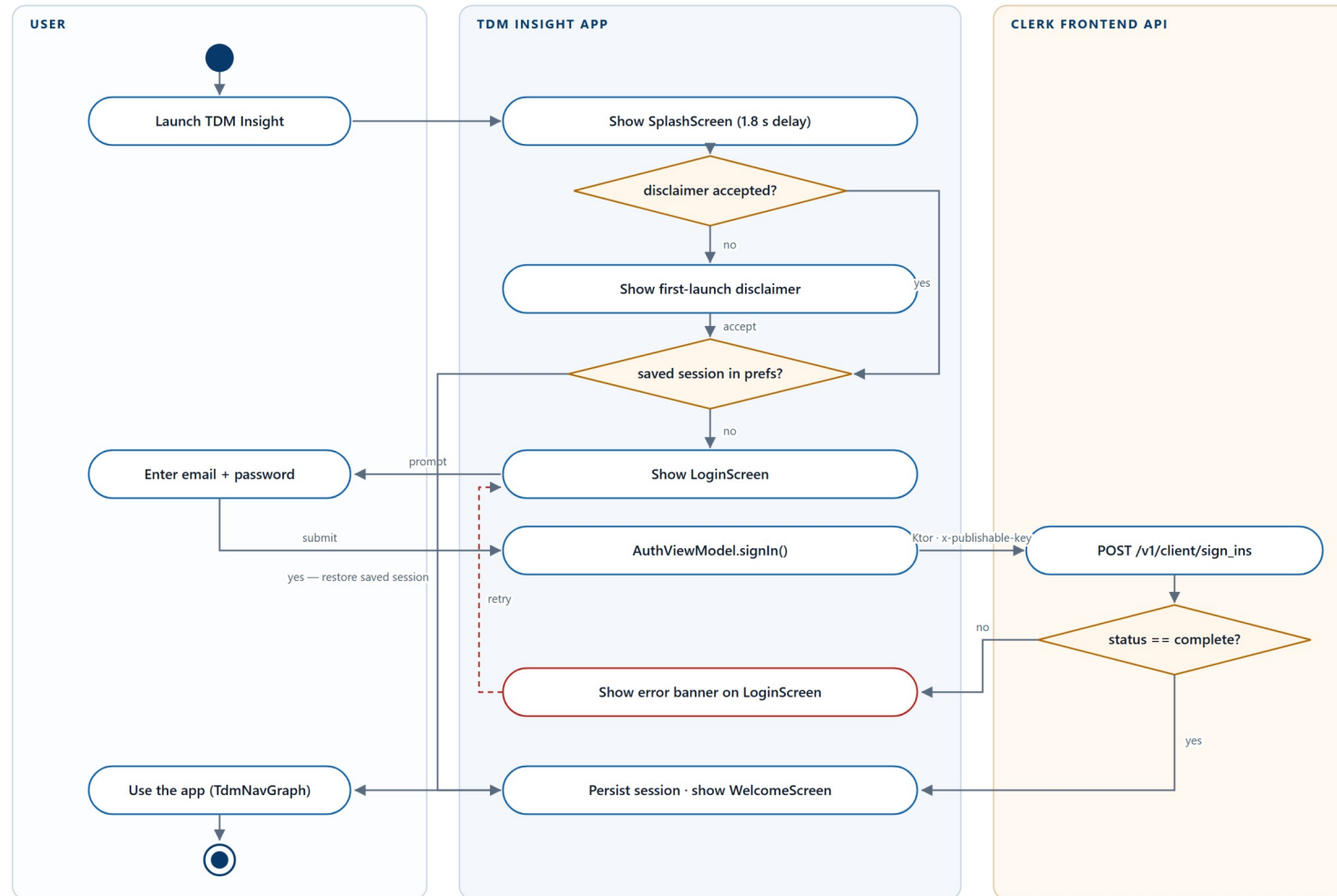
Use case model



Solid line = association · dashed open arrow = «include» / «extend» dependency. The four amber and white ellipses on the right are sub-use-cases invoked by the system rather than started directly by the user. Actors outside the boundary form the system environment; the two external systems are reached only through the Ktor (Clerk) and postgres-kt (Supabase) clients.

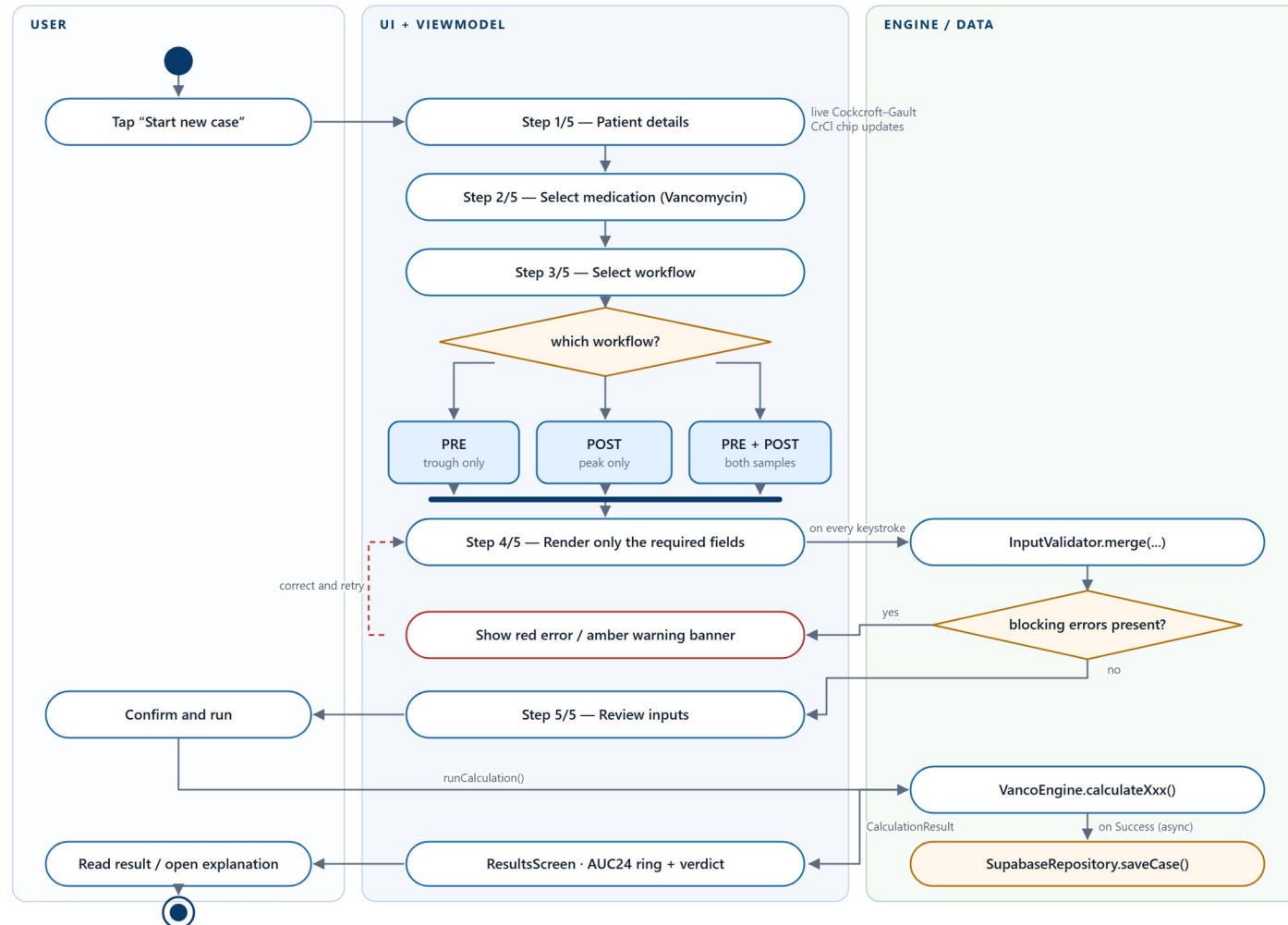
Two human actors, two external system actors, thirteen use cases

Activity — entry and authentication



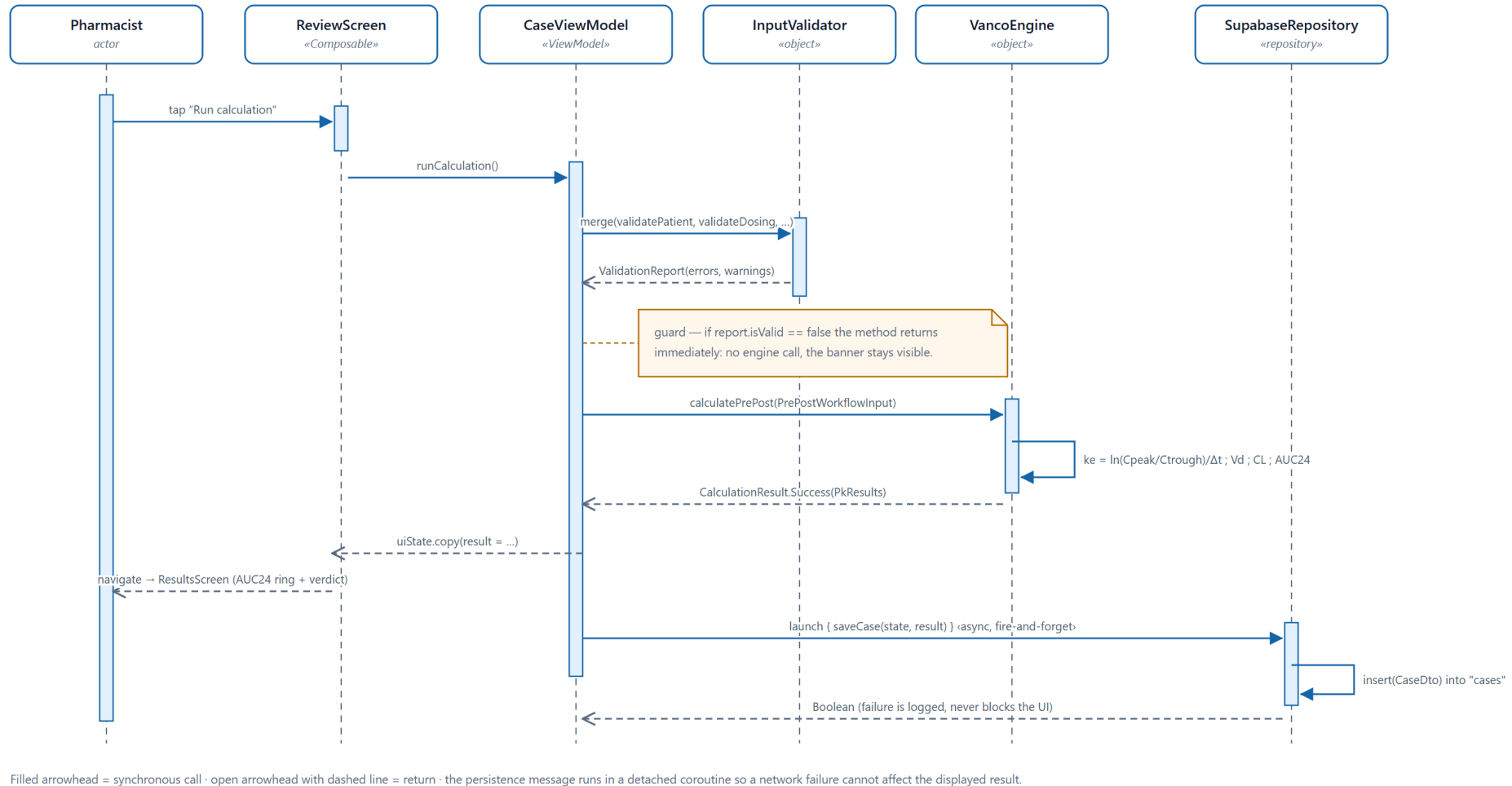
The eight-branch gate in MainActivity, from splash to navigation graph

Activity — the five-step calculation workflow

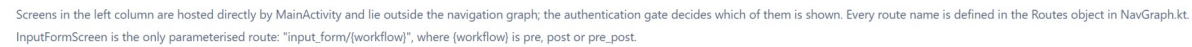


The selected workflow decides which fields are rendered and which engine function runs

Sequence — running and persisting a calculation

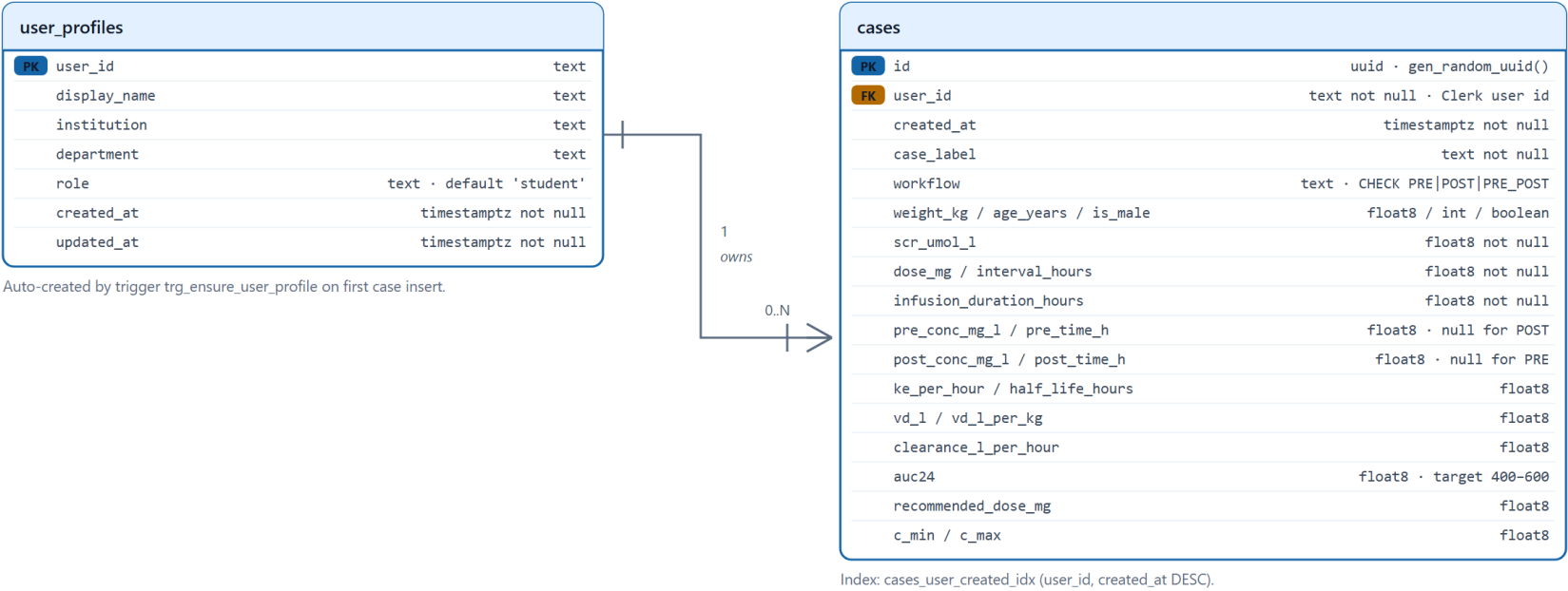


Validation precedes the engine call; persistence is dispatched after the result is displayed



TDM Insight · CDE2313 · Albukhary International University

Database design



Logical relationship: `cases.user_id` and `user_profiles.user_id` both hold the Clerk user id. No FOREIGN KEY constraint is declared in `schema.sql`; the link is maintained by the `ensure_user_profile()` trigger and by the client query.

ACCESS CONTROL AS IMPLEMENTED (supabase/schema.sql · update_v2.sql)

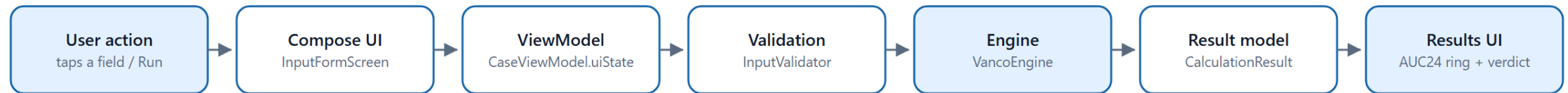
- Row Level Security is ENABLED on both tables.
- Active policies are the permissive fallback set — `anon_read_all` (SELECT using true) and `anon_insert` (INSERT with check true).
- Per-user server-side policies keyed on the Clerk JWT `sub` claim are written but left commented out in `update_v2.sql`, pending a Clerk JWT template.
- Isolation is therefore enforced at query level: `SupabaseRepository.loadRecentCases()` always applies `eq("user_id", userId)`.
- Only the Supabase anon key ships in the APK; the `service_role` key is never present in the client.

The application performs INSERT and SELECT only. There is no UPDATE or DELETE path from the Android client, so a saved case is an append-only audit record.

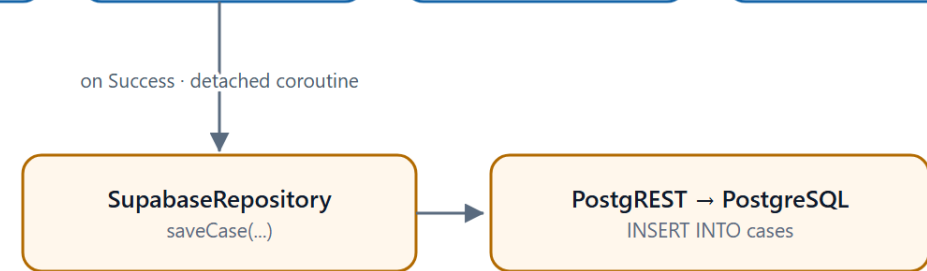
Two tables, one index, one trigger and five Row Level Security policies

End-to-end data flow

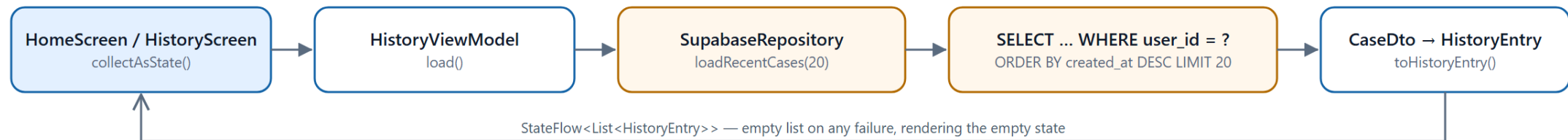
SYNCHRONOUS, ON-DEVICE PATH — no network is required to produce a result



ASYNCHRONOUS PERSISTENCE BRANCH — failure is logged and swallowed



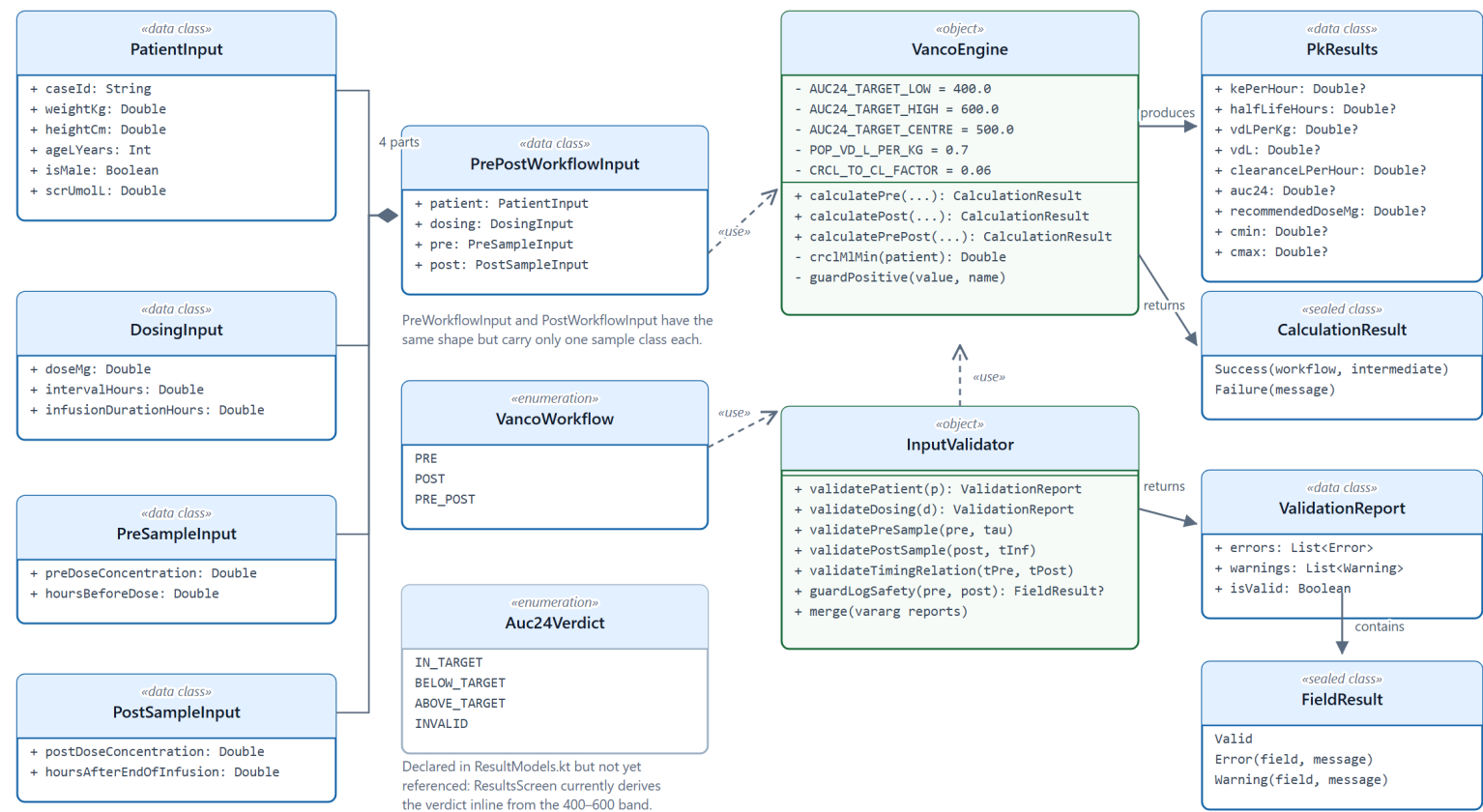
HISTORY READ PATH



The calculation path never blocks on the network. If Supabase or Clerk credentials are absent, the engine, validation and results screens still function; only persistence and the history list are unavailable.

Three paths with different reliability characteristics — only the top one must never fail

Domain class model



Filled diamond = composition · dashed open arrow = «use» dependency · green classes are the framework-free domain layer under unit test.

Immutable data classes and sealed result types; the compiler forces failure to be handled

Three pharmacokinetic pathways

One function per pathway, all returning the same sealed result type.

Pre-dose only

Cockcroft–Gault + population Vd. CrCl estimates clearance; Vd is the 0.7 L/kg prior. Least individualised of the three.

Post-dose only

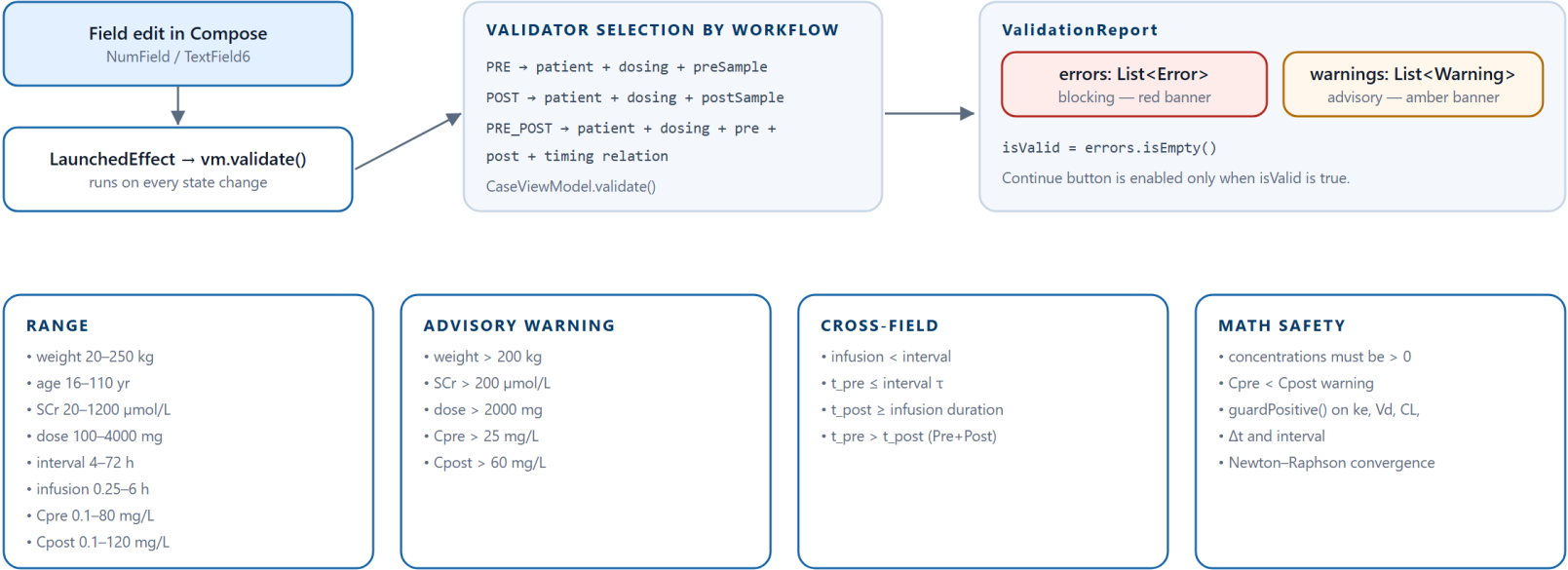
Newton–Raphson fit. Solves the one-compartment infusion model for ke from a single measured concentration, $k_0 = 0.3$, at most 50 iterations.

Pre + Post

Sawchuk–Zaske two-point. Derives both ke *and* an individualised Vd from the patient's own two samples. Preferred when both are available.

$$\begin{aligned} k_e &= \ln(C_{\text{peak}} / C_{\text{trough}}) / (t_{\text{pre}} - t_{\text{post}}) && \# \text{ Pre + Post} \\ V_d &= D \times (1 - e^{-k_e T}) / (k_e \times T \times C_{\text{peak}}) \\ CL &= k_e \times V_d \\ AUC_{24} &= (Dose / \tau) \times 24 / CL && \text{target: 400-600 mg}\cdot\text{h/L} \\ Dose_{\text{recommended}} &= (500 \times CL \times \tau) / 24 \end{aligned}$$

Validation is part of the type system



Every rule above is implemented in data/validation/InputValidator.kt and covered by the 20 JUnit cases in InputValidatorTest.kt; the engine additionally re-runs the same validators before any arithmetic, so a calculation can never be performed on input that the UI has not already cleared.

ValidationReport carries errors and warnings in separate lists; isValid = errors.isEmpty()

The case study requires that an error be distinguishable from information that merely needs review. Making that a **type** distinction, not a rendering choice, means warnings can never accidentally block a calculation.

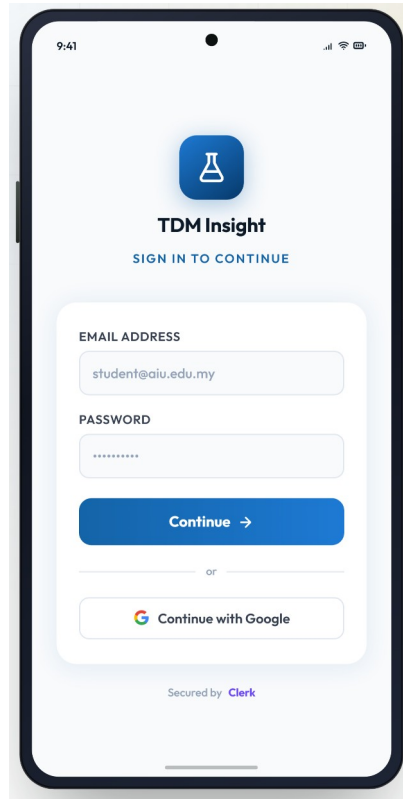
Worked examples — one per pathway

Computed with the engine's exact arithmetic. All three patients are fictional.

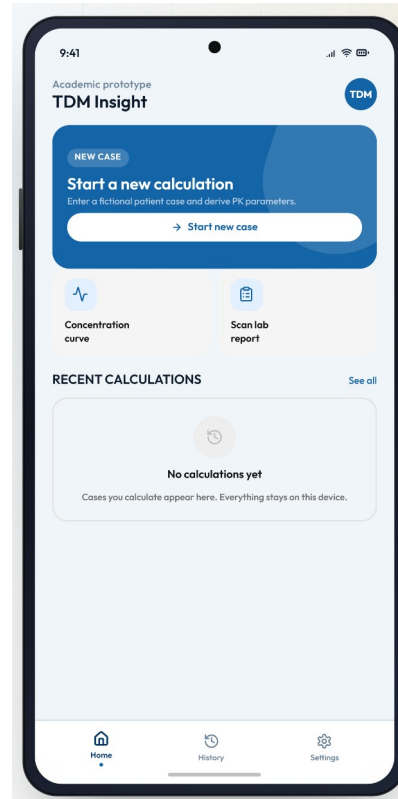
	Pre-dose only	Post-dose only	Pre + Post
Patient	72 kg, 50 y, male, SCr 88	70 kg, 60 y, male, SCr 95	68 kg, 62 y, male, SCr 98
Regimen	1000 mg q12h over 1 h	1000 mg q12h over 1 h	1000 mg q12h over 1 h
Samples	Trough 10.2 mg/L	Peak 15.0 mg/L at 4 h	Peak 26.0 at 2 h; trough 12.5 at 10.5 h
ke (h ⁻¹)	0.1076	0.0881	0.0862
t½ (h)	6.44	7.87	8.04
Vd (L)	50.4 (0.70 L/kg, population)	49.0 (0.70 L/kg, population)	36.9 (0.54 L/kg, derived)
CL (L/h)	5.425	4.315	3.175
AUC ₂₄	368.7 — below target	463.5 — in target	629.9 — above target
Recommended	1356 mg q12h	1079 mg q12h	794 mg q12h

The Pre + Post column shows why two samples are better: the derived Vd of 0.54 L/kg is well below the 0.7 L/kg prior, so the single-point pathways would have understated this patient's exposure.

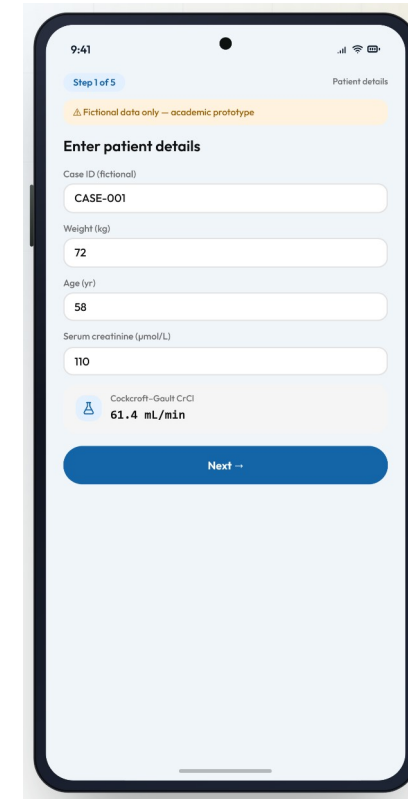
Authentication, home and patient details



Sign in — Clerk

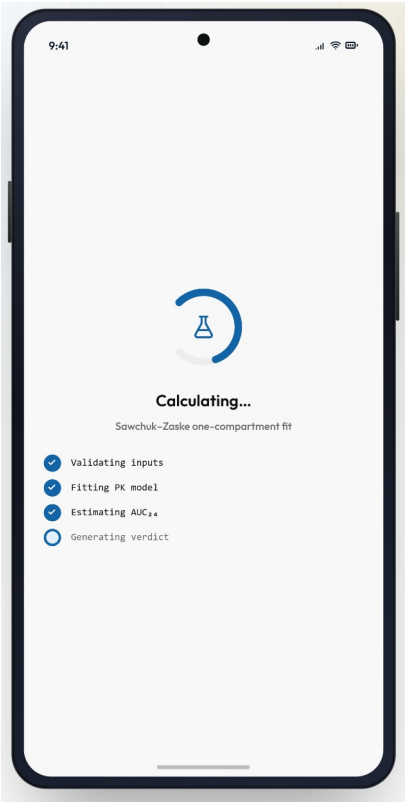


Home — new case + recent

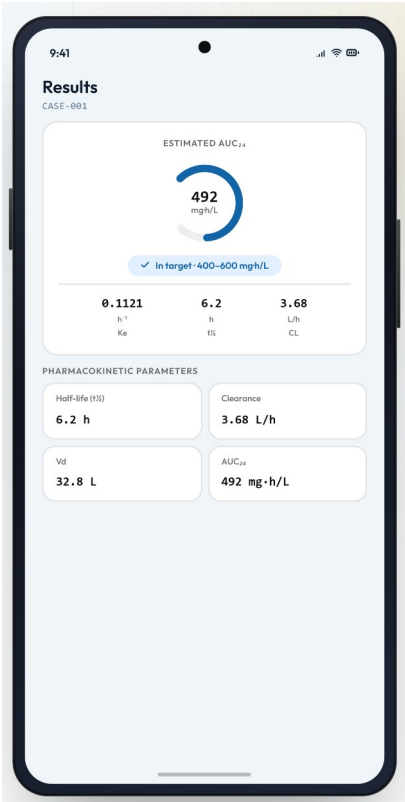


Step 1/5 — live CrCl chip

Calculating, results and explanation



Fitting the model



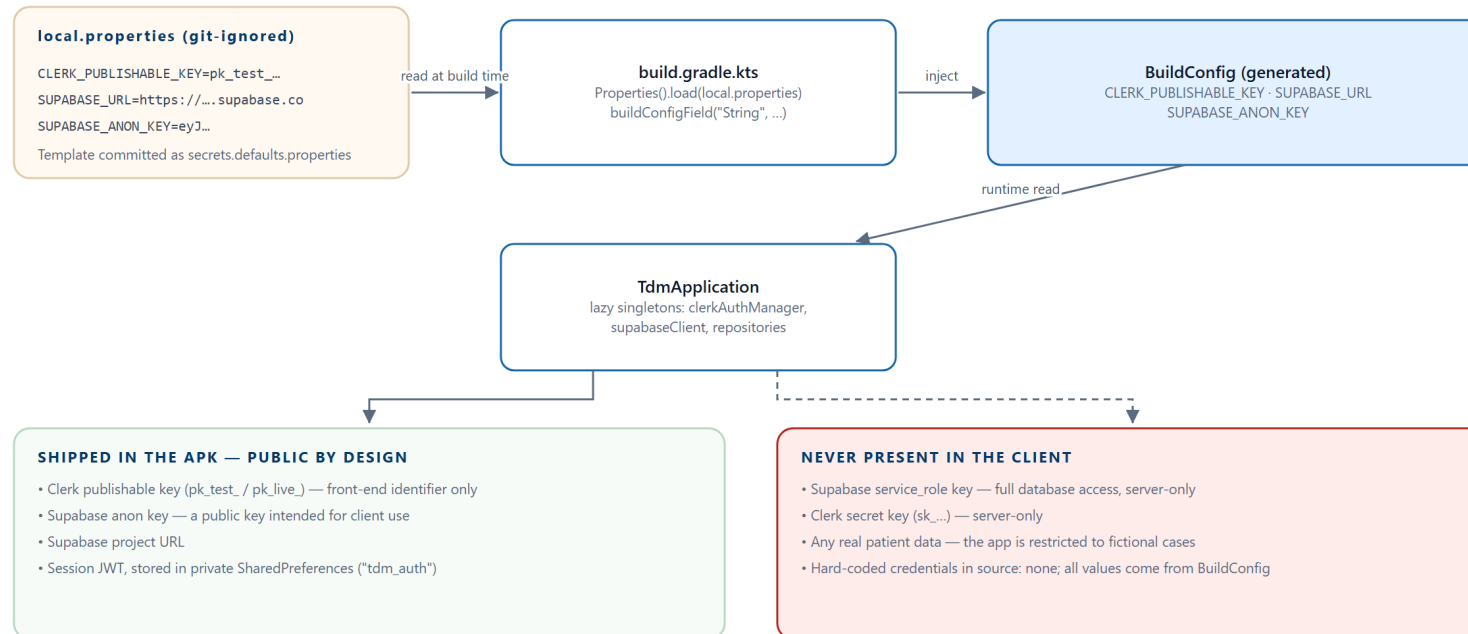
AUC₂₄ ring + verdict

The screen is titled "How was this calculated?" and lists four phases:

- 1 INPUT VALUES**: CASE-001 - 72 kg - 58 yr - Male, Scr 110 µmol/L - Dose 1250 mg q12h
- 2 KE DERIVATION (SAWCHUK-ZASKE)**: $k_e = \ln(C_{peak}/C_{trough}) / \Delta t = \ln(18.2/8.4) / 6 = 0.1121 \text{ h}^{-1}$
- 3 PK PARAMETERS**: $t_{1/2} = 6.2 \text{ h}$, $V_d = 32.8 \text{ L}$, $CL = 3.68 \text{ L/h}$, $AUC_{24} = 492 \text{ mg}\cdot\text{h/L}$
- 4 FINAL RESULT**: $AUC_{24} = 492 \text{ mg}\cdot\text{h/L} \rightarrow \text{In target}$, Recommend: 1250 mg every 12 h

Four-phase explanation

Authentication and credential model



Credentials reach the app through BuildConfig; nothing sensitive is committed

Only public keys ship in the APK. The Supabase service_role key and the Clerk secret key are never referenced anywhere in the project. The app declares one permission: INTERNET.

04

Testing, honesty and next steps

What we verified, what we found, and what remains.

Testing

Effort was concentrated where defects would be most costly and least visible.

31

JUnit unit tests

11

engine tests

20

validator tests

14

manual end-to-end scenarios

- Every **blocking rule** has a test asserting the calculation is refused — a validator that silently passes bad input is worse than none.
- Several tests assert a value produces a **warning and not an error**, because collapsing the two tiers would break an explicit case-study requirement.
- `engine never produces NaN on edge inputs` exists because a NaN would render as a plausible-looking dash rather than an obvious failure.
- Manual scenarios cover all three workflows, rotation, back-navigation, theme switching, sign-out and offline use.

A defect we found — and are reporting

Discovered while verifying the worked examples for the report.

- The one-compartment model has a ceiling: as $k_e \rightarrow 0$, the modelled concentration tends to **Dose ÷ Vd_{population}**.
- When the **measured** peak exceeds that ceiling, no positive root exists. Newton–Raphson drives k_e down, `coerceAtLeast(1e-6)` clamps it, two iterations compare equal — and the loop declares convergence.
- `guardPositive` passes because $1e-6 > 0$, so the engine returns **Success** with $CL \approx 4.6 \times 10^{-5}$ L/h and **AUC₂₄ $\approx 4 \times 10^7$ mg·h/L**.

```
68 kg patient, 1000 mg over 1 h, measured 22.5 mg/L  
Vd = 0.7 x 68 = 47.6 L      ceiling = 1000 / 47.6 = 21.0 mg/L < 22.5
```

The existing test post workflow handles plausible two-point-like data uses exactly these values and **passes** — it asserts only that the result is a **Success**. A weak assertion concealed the defect. Fix: reject the input before iterating, and treat a clamped k_e as non-convergence.

Limitations we are not hiding

Not clinically validated

An academic prototype. The equations have not been validated for patient care and must never drive a real prescription.

Post-dose defect

Degenerate fit above the model ceiling. The Pre and Pre + Post pathways are unaffected.

Database isolation

Per-user RLS policies are written but commented out; isolation is currently enforced by the client query.

Model scope

One compartment, fixed 0.7 L/kg prior for the single-sample pathways.

Three inert controls

Concentration-curve tile, lab-report scan row and share icon are placeholders for optional enhancements.

No Compose UI tests

Dependencies are declared but the instrumented source set is empty.

Future work

Priority	Item	Why it is tractable
1	Correct the Post-dose fit and add a regression test	A ceiling check before the loop; a few lines
2	Enable per-user Row Level Security	Policies are already written; <code>getSupabaseToken()</code> already exists
3	Wire the concentration–time graph	<code>PkCurveGraph</code> is already implemented as a Canvas Composable
4	Camera capture with confirm-before-use OCR	Two interface entry points already exist
5	Export or share a calculation summary	Replaces the inert share icon
6	Compose UI tests for the wizard	Dependencies already declared
7	Extend to a second medication	One function per pathway; no architectural change needed

Every mandatory requirement, traced

Case study requirement	Implemented in	Status
Native Android · Kotlin · Compose · Material 3	<code>Kotlin_app/</code> · <code>compileSdk 35</code> · <code>minSdk 26</code>	Met
Pre, Post and Pre + Post workflows	<code>VancoEngine</code> — three public functions	Met
Dynamic input forms	<code>InputFormScreen</code> conditional field groups	Met
Input and cross-field validation	<code>InputValidator</code> — 7 public functions	Met
Engine separated from the UI	<code>domain/engine/</code> — no Android imports	Met
Intermediate and final PK results	<code>PkResults</code> — nine values	Met
Explainable results	<code>ExplanationScreen</code> — four phases	Met
Material 3 light and dark mode	<code>Color.kt</code> + <code>Theme.kt</code> + Settings selector	Met
Mandatory academic disclaimer	First-launch gate · banners · About screen	Met

Repository and responsible AI

Repository structure

Follows the prescribed layout: `Kotlin_app/`, `docs/`, `apk/`, `landingPage/`, with a README carrying all thirteen required sections.

Credentials

`local.properties` is git-ignored; `secrets.defaults.properties` is the committed template. No key has ever entered version history.

Commits

Scoped to one logical change, with conventional prefixes. **Authorship still needs distributing across all three members before submission.**

AI — where it helped

Learning Compose APIs, drafting layouts, a refactor, and one debugging session. Output was reviewed and frequently modified.

AI — where it did not

No clinical formula, constant or target came from a language model. Every equation is cited and commented with its source.

AI — full disclosure

The drafting of the project report surfaced the Post-dose defect. We reported it rather than removing the example.

Conclusion

- TDM Insight turns a paper worksheet into a **guided, validated and explainable** mobile workflow — all nine mandatory requirements are implemented.
- The **calculation engine is framework-free** and covered by 31 unit tests. Remove the network layer entirely and it still compiles and passes.
- Two optional enhancements — identity and persistent history — were added **without compromising the core**. No credentials? The calculator still works.
- We document our own defects. A weak assertion hid a bug in the Post-dose fit; it is reported in full, with a reproduction and a fix.

For a system that models a clinical calculation, that honesty is not incidental — it is the difference between a teaching aid and a liability.

References

Burden, R. L., & Faires, J. D. (2010). Numerical analysis (9th ed.). Brooks/Cole.

Cockcroft, D. W., & Gault, M. H. (1976). Prediction of creatinine clearance from serum creatinine. *Nephron*, 16(1), 31–41.

Google. (2024). Guide to app architecture. Android Developers. <https://developer.android.com/topic/architecture>

Mohd Zaki, M. Z. (2026). CDE2313 project case study description: TDM Insight [Course case study]. Albukhary International University.

Rybak, M. J., Le, J., Lodise, T. P., et al. (2020). Therapeutic monitoring of vancomycin for serious methicillin-resistant *Staphylococcus aureus* infections: A revised consensus guideline. *American Journal of Health-System Pharmacy*, 77(11), 835–864.

Sawchuk, R. J., & Zaske, D. E. (1976). Pharmacokinetics of dosing regimens which utilize multiple intravenous infusions. *Journal of Pharmacokinetics and Biopharmaceutics*, 4(2), 183–195.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Supabase. (2024). Row Level Security. <https://supabase.com/docs/guides/database/postgres/row-level-security>

Thank you

Questions, and a live demonstration of all three workflows.

TDM Insight · CDE2313 Mobile Application Development · Albukhary International University

Academic prototype — all demonstration cases are fictional.